AndMTProto · UI proposal B

Dashboard: guided setup and a network view

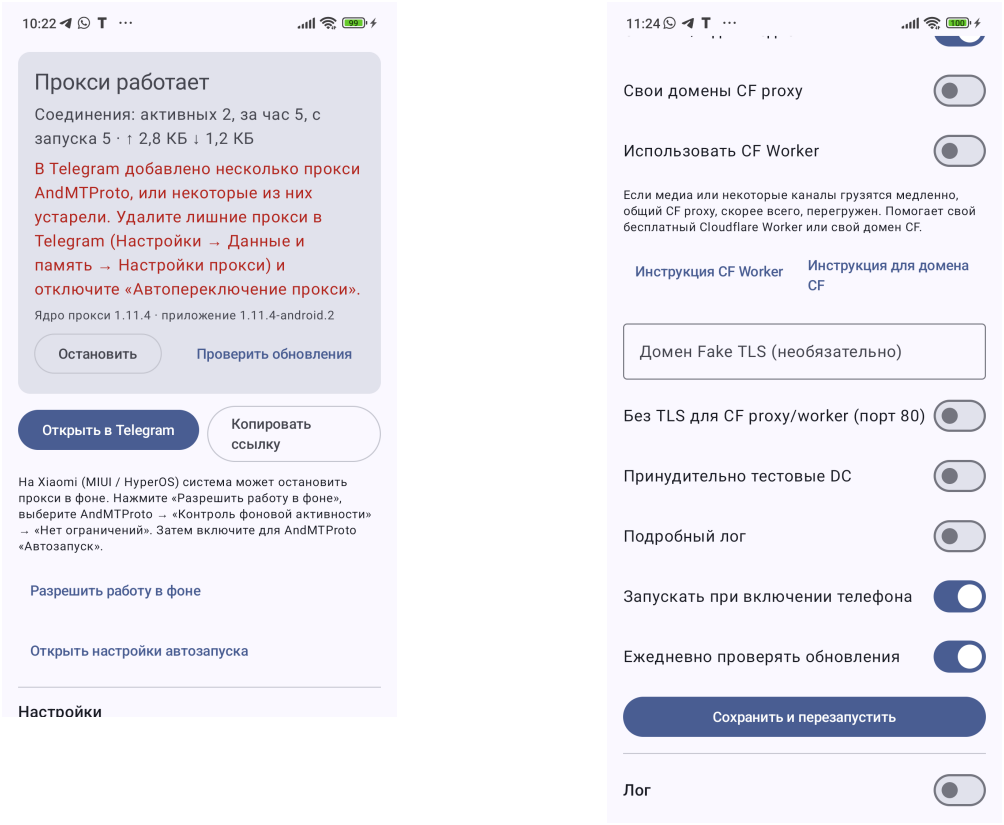
Turn the app into a small dashboard with bottom navigation, a first-run wizard, per-data-center route health and a one-tap self-test.

Scope	New navigation (4 tabs), setup wizard, Home dashboard, Network tab with self-test, network profiles in Settings.
Effort	About 3–4 weeks, including adapter work for per-DC status and the self-test.
Risk	Medium. New navigation model and new data from the adapter; more strings to translate and test.
Best for	Making AndMTProto self-diagnosing, so users fix route problems without sending logs.

Prepared 7 November 2026 for the AndMTProto maintainer. Baseline: AndMTProto 1.11.4-android.3 (main, with the daily update check from PR #19). Screens in this document are wireframe mockups; labels are in Russian because that is the app's primary language.

1. Where the UI stands today

The whole app is one long scrolling screen built with Jetpack Compose and Material 3 (dynamic colours on Android 12+). From top to bottom: the status card, the Telegram buttons, a block of background-work hints, about twenty settings, the Save button and the log. It works, and every recent feature found a place on it, but it has grown by accretion. The screenshots below are from the test phone (Redmi Note 10 Pro, MIUI 14) on 7 November 2026.



Left: status card with the stale-proxy hint. Right: the lower half of the settings. (The secret field is cropped out.)

Problems worth fixing

Problem	Effect on users
No single glanceable state. The status is a text title; Start/Stop is a small button next to “Проверить обновления”.	Users can't tell at a glance whether Telegram is protected; the most important action has the same weight as a secondary one.
Settings and status share one screen. Twenty-plus fields, most of them advanced (pool size, buffer, test DCs, no-TLS).	The everyday screen is long and intimidating; mistakes in advanced fields are easy (e.g. a non-loopback address).
The secret is shown in plain text on the main screen.	Any screenshot of the main screen shared for support leaks it, and with it access to the proxy.
Hints are walls of text (MIUI background work, CF overload, stale proxies).	They push the controls down, stay after the problem is solved and get ignored.
Route health is invisible. Whether DC2 goes over WebSocket or Cloudflare, or the shared CF proxy fails, is only in the log.	When Telegram is slow, users can't see why; support needs log screenshots.

Problem	Effect on users
Setup is spread out. Start → open in Telegram → battery → MIUI autostart are separate buttons with no sense of progress.	First-run users miss the background-work steps, then the proxy dies in the background.
Layout shifts after Save and after Stop (seen while testing).	Mis-taps: one landed on “Открыть в Telegram”.

2. What modern Android UI looks like in 2026

Platform and Material guidance

- **Material 3 Expressive** extends Material 3 with spring-based motion, shape morphing, emphasised type and new components. Google's guidance is Compose-first. Caveat: much of the expressive API was still alpha or experimental in Compose Material3 in 2026, so a proposal should use it only where it degrades gracefully.
- **Edge-to-edge is mandatory** for apps targeting Android 16 (AndMTProto targets API 36), and **predictive back** animations are on by default. The UI must handle insets and back gestures properly; a multi-screen design should use the navigation components that support predictive back.
- **Progress-centric notifications** (Notification.ProgressStyle, the basis of Live Updates) exist since Android 16. They aren't a natural fit for an always-on proxy, but the foreground notification can carry richer state (route, traffic).
- **Dynamic colour** stays the norm on Android 12+, with a brand palette as fallback; AndMTProto already does this.

Patterns from VPN and proxy apps

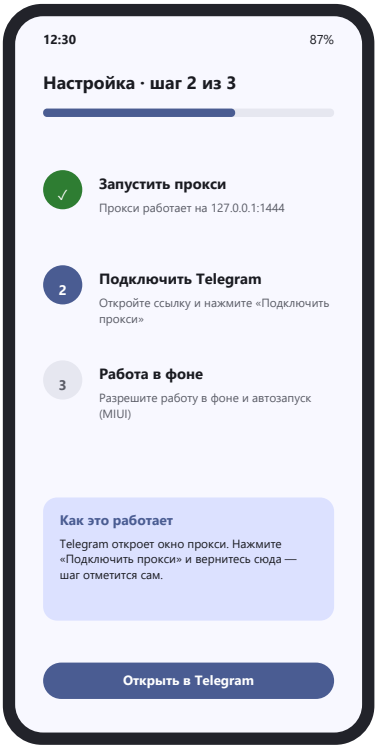
- One unmistakable **connect control** in the centre, with a clear connected / disconnected state and simple stats beside it.
- **Explicit transitional states** ("Подключение…") with animation, never a status that misrepresents the real connection. AndMTProto already distinguishes "starting" from "listening"; the UI should show it.
- Live **timer and traffic** readouts reassure users that it works.
- Redesigns (e.g. NordVPN 2025) concentrated on the connection screen and first-time use rather than on settings.

Constraints specific to AndMTProto

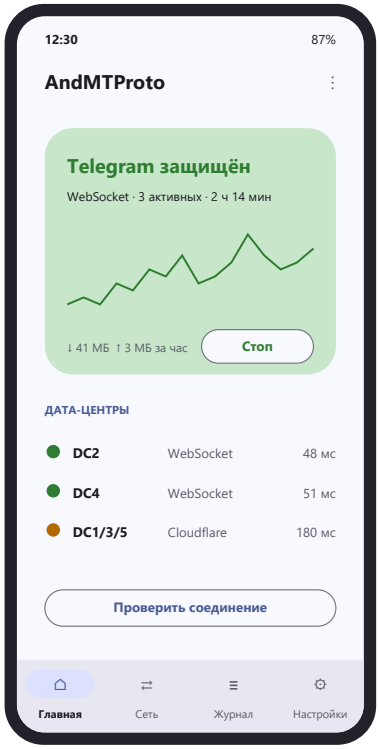
- Users are often non-technical and on Xiaomi phones, where background restrictions are the main failure; the UI has to guide them through MIUI settings.
- Russian first, English second; Russian strings are 20–30 % longer, so layouts need room.
- Play Protect passed review; the redesign must not add permissions, trackers or network calls.
- The Python core is upstream code: new UI data must come from the adapter (android_entry.py), never from patching proxy/.

3. The proposal

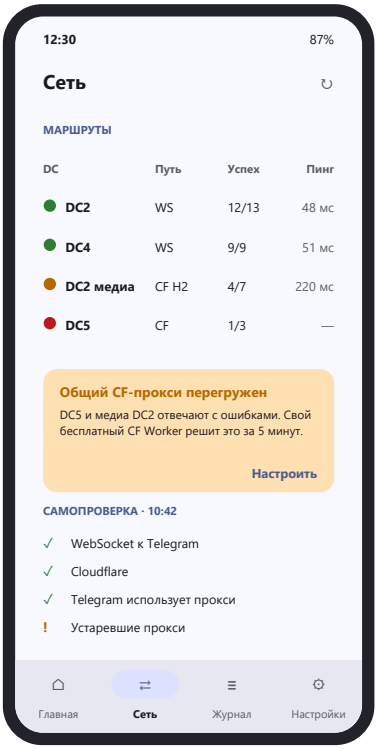
Proposal B treats AndMTProto as a small network tool rather than a switch. A **bottom navigation bar** (Главная, Сеть, Журнал, Настройки) separates watching, diagnosing and configuring. A **setup wizard** takes new users from install to a working, background-safe proxy, ticking steps off automatically from real state.



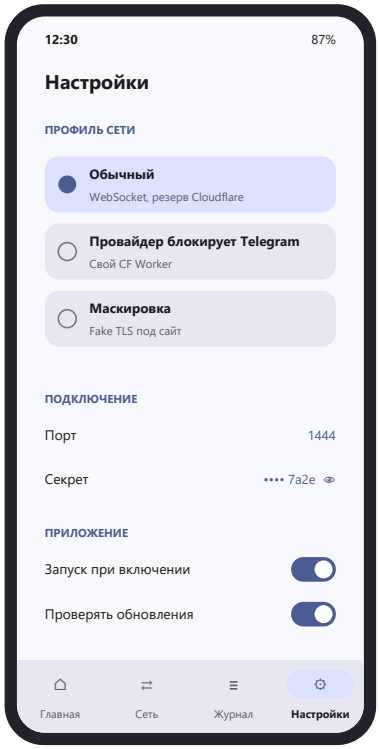
1 First-run wizard



2 Home dashboard



3 Network and self-test



4 Settings with profiles

Screen by screen

Screen	What it does
1 Wizard	Three steps with progress: start the proxy; connect Telegram (ticks itself when the first good handshake arrives); allow background work (ticks when the battery exemption is granted; MIUI autostart is confirmed by the user). Shown on first launch and reachable later from the menu.
2 Home	A state card (green / amber / red) with uptime, a one-hour traffic sparkline and Stop. Below, data centres with their current route and latency. “Проверить соединение” runs the self-test.
3 Network	Route table per DC: WebSocket / CF / CF H2, success ratio of the last attempts, latency. Context cards explain problems in plain words (“общий CF-прокси перегружен”) with a fix. The self-test checks WebSocket and Cloudflare reachability, whether Telegram uses the proxy, and stale proxy entries (the rejected-handshake counter).
4 Settings	Network **profiles** set the right group of options at once: Обычный, Провайдер блокирует Telegram (own CF Worker), Маскировка (Fake TLS). Individual options remain below for experts; the secret is masked.
Log tab	Full-screen log with level filters, search, Copy and Share (secret redacted).

4. Data the UI needs from the adapter

B shows information that today exists only inside the Python core or in the log. It must come from android_entry.py, without touching upstream code:

Data	Source	Work
Route per DC (WS / CF / H2) and success ratio	Count route decisions per DC in the adapter, e.g. through a logging filter on the core's routing log lines, or from stats fields where they exist.	Medium
Latency per DC	Time the WebSocket handshake for pool connections; the core logs durations of closed sessions.	Medium
Self-test	New adapter function: connect to kws endpoints and a CF domain with short timeouts, return a result list; reuse the core's TLS context (X25519 rule).	Medium
Telegram connected	First accepted handshake since start (counters already there).	Small
Stale proxy entries	bad_recent (added in PR #18).	None
Uptime and traffic history	Existing 10-second samples; keep bytes too.	Small

Parsing log lines is fragile across upstream updates, so the sync checklist in android/README.md would gain an item to re-check these hooks; an adapter test can pin the log formats it relies on.

5. Interaction and visual details

- **Navigation:** Material 3 NavigationBar with four destinations; navigation-compose for predictive back; state survives tab switches.
- **State colour** carries meaning everywhere: green (all routes WebSocket), amber (some on Cloudflare or slow), red (proxy down or Telegram not using it).
- **Expressive motion:** the state card morphs between states; the wizard advances with shared-axis transitions; all motion respects the system “remove animations” setting.
- **Accessibility:** the sparkline has a text equivalent (“41 МБ за час”); the DC table is a list with full sentences for TalkBack.
- **Large screens:** on tablets and foldables the NavigationBar becomes a NavigationRail and Home + Network sit side by side.

6. Implementation plan

Phase	Work	Days
1	Navigation scaffold, four tabs, move existing screens in (log, settings).	2
2	Adapter: per-DC route counters, latency, self-test function, status() fields, tests.	4–5
3	Home dashboard: state card, sparkline, DC list.	2–3
4	Network tab: route table, context cards, self-test UI.	2–3
5	Wizard with automatic step detection; profiles in Settings.	3
6	Strings RU/EN, accessibility, MIUI phone test, README and release notes.	2–3

Risks

- Adapter hooks that depend on the core's log format can break on upstream updates; mitigated by tests and the sync checklist.

- Self-test traffic is new network activity; it must run only on demand, never in the background, to keep Play Protect and the README's privacy statement true.
- More UI surface means more strings and more to test on MIUI.

7. How it compares with proposal A

	A · Evolution	B · Dashboard
Navigation	One screen + Settings	Bottom bar, four tabs
First-run guidance	Attention cards	Step-by-step wizard
Route visibility	Two chips	Per-DC table, latency, self-test
Python adapter changes	None (optional status fields)	New per-DC status and self-test
Effort	1–2 weeks	3–4 weeks
Risk	Low	Medium

When to choose B: if most support time goes to “Telegram is slow, why?”. The Network tab and self-test answer that without logs. B can also be reached in two steps: ship A, then add the Network tab and wizard.

Sources

Android Developers Blog — Android 16 is here (edge-to-edge, predictive back, ProgressStyle)
https://android-developers.googleblog.com/2025/06/android-16-is-here.html

Material 3 skill wiki (community summary of Material 3 Expressive / I/O 2026 guidance)
https://github.com/hamen/material-3-skill/wiki

Material Design 3 overview (third-party summary)
https://aksite.pages.dev/ai-native-design/material-design/

VPN connection switch UI notes (VP0; promotional, used as a starting point)
https://vp0.com/blogs/vpn-connection-switch-ui-clone-ios

HeroVPN app design case study
https://contra.com/p/gvr23BQg-hero-vpn-secure-vpn-app-design

VPNIFY UX teardown
https://screensdesign.com/showcase/vpnify-unlimited-vpn

Bitmask VPN visual design proposal
https://0xacab.org/leap/bitmask-vpn/-/issues/513

Tom's Guide — NordVPN mobile app redesign
https://www.tomsguide.com/computing/vpns/nordvpn-announces-mobile-app-redesign-heres-what-you-need-to-know

Material 3 Expressive details come from secondary summaries; check the AndroidX Compose Material3 release notes for the exact API status before implementation.